

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



DỮ LIỆU LỚN (CO3137)

BÁO CÁO BÀI TẬP LỚN

PHÂN TÍCH CẢM XÚC
MẠNG XÃ HỘI
THEO THỜI GIAN THỰC

GVHD: TS. Nguyễn Quang Hùng

SV thực hiện:	Nguyễn Đức Trung Kiên	2311734
	Bùi Quang Huy	2311124
	Trần Quốc Cường	2310405
	Huỳnh Kim Quý	2312894

Danh sách phân công nhiệm vụ

STT	Họ và tên	Nhiệm vụ	Tỉ lệ hoàn thành
1	Nguyễn Đức Trung Kiên	BFF, dashboard và viết báo cáo	100%
2	Bùi Quang Huy	Postgre database	100%
3	Trần Quốc Cường	Flink, model và viết báo cáo	100%
4	Huỳnh Kim Quý	Data và setup pipeline	100%

Mục lục

1	Giới thiệu	3
1.1	Bối cảnh	3
1.2	Vấn đề đặt ra	3
1.3	Nghiên cứu liên quan	3
1.4	Giải pháp đề xuất	3
2	Cơ sở lý thuyết	4
2.1	Kiến trúc tổng quan (High-level design)	4
2.2	Kiến trúc chi tiết (Low-level design)	4
2.2.1	Kafka Topics và Schema	4
2.2.2	Apache Flink Job	5
2.2.3	Database Schema	5
2.2.4	BFF API Endpoints	5
2.3	Mô hình RoBERTa	5
2.3.1	Giới thiệu RoBERTa	5
2.3.2	Model được sử dụng	5
2.3.3	Quy trình inference	6
3	Giải pháp và triển khai	7
3.1	Môi trường và công cụ	7
3.1.1	Hạ tầng	7
3.1.2	Công cụ phát triển	7
3.2	Data Ingestion	7
3.2.1	Producer	7
3.2.2	Worker	7
3.3	Xử lý luồng với Apache Flink	7
3.4	Phân tích cảm xúc	8
3.5	BFF và Dashboard	8
3.5.1	Backend For Frontend (BFF)	8
3.5.2	Dashboard React	8
3.5.3	Confidence Filter	8
3.6	Kết quả demo	9
3.6.1	Khởi động hệ thống	9
3.6.2	Kết quả	9
4	Hướng phát triển	10
4.1	Mở rộng quy mô (Horizontal Scaling)	10
4.2	Tích hợp nguồn dữ liệu thực	10
4.3	Cải thiện hiển thị — Line Chart theo thời gian thực	10
4.4	Sửa lỗi và chuẩn hóa thời gian	11
4.5	Dự báo xu hướng bằng AI	11
4.6	Tăng cường bảo mật và vận hành	11
	Tài nguyên và Mã nguồn	12

1 Giới thiệu

1.1 Bối cảnh

Trong thời đại số hóa hiện nay, mạng xã hội và các sàn thương mại điện tử như Amazon tạo ra hàng triệu đánh giá sản phẩm mỗi ngày. Việc phân tích cảm xúc (sentiment analysis) từ các đánh giá này giúp doanh nghiệp nắm bắt nhanh phản hồi của khách hàng, từ đó đưa ra quyết định kinh doanh kịp thời. Tuy nhiên, với khối lượng dữ liệu lớn và tốc độ sinh ra liên tục (velocity), các phương pháp xử lý truyền thống theo batch không đáp ứng được nhu cầu phân tích theo thời gian thực.

1.2 Vấn đề đặt ra

Các hệ thống phân tích cảm xúc hiện tại thường gặp những hạn chế sau:

- **Độ trễ cao:** Xử lý batch theo giờ hoặc ngày khiến thông tin không phản ánh được tình trạng hiện tại.
- **Thiếu khả năng mở rộng:** Hệ thống đơn lẻ không xử lý được khi lượng dữ liệu tăng đột biến.
- **Độ chính xác thấp:** Các phương pháp dựa trên từ điển hoặc mô hình đơn giản không nắm bắt được ngữ cảnh phức tạp.
- **Không có giao diện trực quan:** Kết quả phân tích khó theo dõi và giám sát theo thời gian thực.

1.3 Nghiên cứu liên quan

Nhiều nghiên cứu đã đề xuất các kiến trúc streaming để xử lý dữ liệu cảm xúc:

- **Lambda Architecture:** Kết hợp batch layer và speed layer, nhưng phức tạp trong vận hành.
- **Kappa Architecture:** Chỉ dùng stream processing, đơn giản hơn nhưng yêu cầu hệ thống message queue mạnh.
- **Apache Kafka + Spark Streaming:** Phổ biến trong công nghiệp nhưng Spark có độ trễ micro-batch.

Đề án này lựa chọn kiến trúc Kappa với Apache Kafka và Apache Flink để đạt độ trễ thấp nhất, kết hợp mô hình RoBERTa cho độ chính xác cao.

1.4 Giải pháp đề xuất

Chúng tôi đề xuất một pipeline xử lý luồng dữ liệu hoàn chỉnh:

1. **Thu thập dữ liệu:** Đọc review từ dataset Amazon và đưa vào Kafka topic.
2. **Tiền xử lý:** Apache Flink làm sạch văn bản (loại URL, @mention, #hashtag).
3. **Phân tích cảm xúc:** Mô hình RoBERTa phân loại positive/neutral/negative với điểm tin cậy.
4. **Lưu trữ:** Kết quả được lưu vào PostgreSQL theo schema chuẩn hóa.
5. **Trực quan hóa:** Dashboard React hiển thị thống kê và bảng review theo thời gian thực.

2 Cơ sở lý thuyết

2.1 Kiến trúc tổng quan (High-level design)

Hệ thống được thiết kế theo mô hình pipeline streaming một chiều, trong đó dữ liệu chảy từ nguồn đến đích mà không cần lưu trữ trung gian lâu dài:

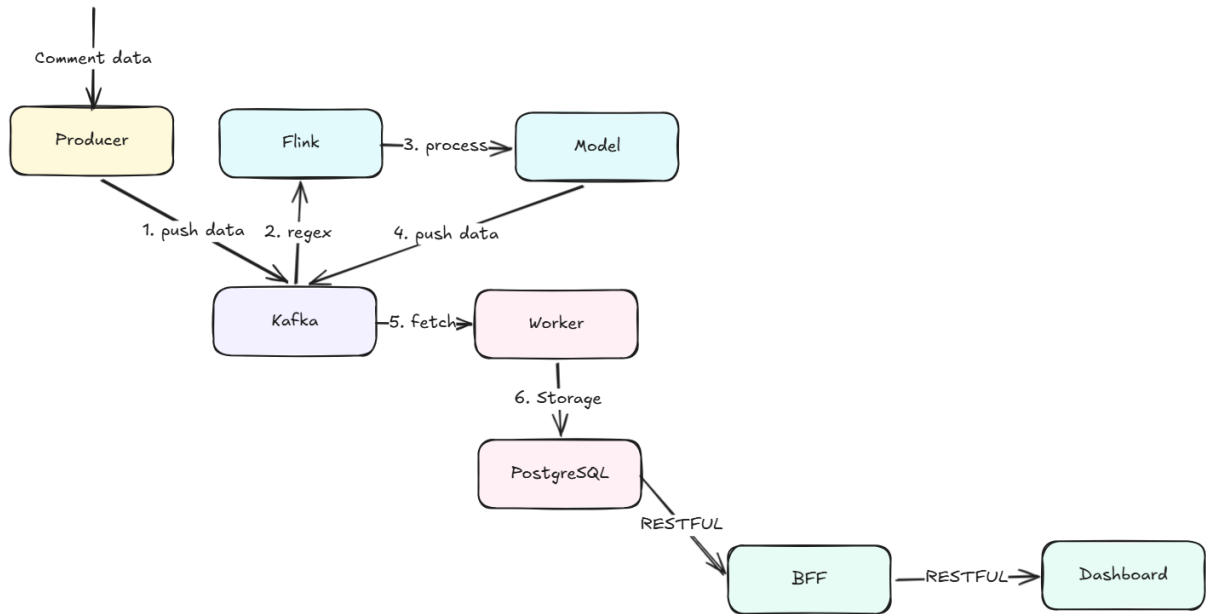


Figure 1: Kiến trúc tổng quan

Các thành phần chính:

- **Apache Kafka:** Message broker phân tán, đảm bảo thông lượng cao và độ bền dữ liệu. Hệ thống dùng 2 topic: `social_media_stream` (raw) và `final_comments` (đã phân tích).
- **Apache Flink:** Stream processing engine xử lý dữ liệu với độ trễ mili-giây, thực hiện tiền xử lý văn bản.
- **RoBERTa Model:** Mô hình ngôn ngữ lớn thực hiện phân loại cảm xúc.
- **PostgreSQL:** Lưu trữ kết quả với schema quan hệ chuẩn hóa.
- **BFF + Dashboard:** Lớp API Express.js và giao diện React cung cấp khả năng trực quan hóa.

2.2 Kiến trúc chi tiết (Low-level design)

2.2.1 Kafka Topics và Schema

Hệ thống sử dụng 3 Kafka topic:

- `social_media_stream`: Nhận raw review từ producer, message là JSON với các trường `text`, `timestamp`, `asin`, `star_rating`.
- `processed_comments`: Nhận output từ Flink sau khi làm sạch văn bản.
- `final_comments`: Nhận output từ model sau khi phân tích cảm xúc, bổ sung `sentiment_label` và `confidence_score`.

2.2.2 Apache Flink Job

Flink job (Main.java) thực hiện:

1. Đọc từ topic `social_media_stream` qua `KafkaSource`
2. Deserialize JSON thành object `SocialMediaComment`
3. Làm sạch văn bản: loại bỏ URL (`http://...`), `@mention`, `#hashtag` bằng regex
4. Serialize và ghi vào topic `processed_comments` qua `KafkaSink`

2.2.3 Database Schema

PostgreSQL lưu trữ theo 5 bảng:

- **AmazonReview**: Thông tin gốc (`review_id`, `product_asin`, `star_rating`, `original_text`, `event_time`)
- **SentimentResult**: Kết quả phân tích (`review_id` FK, `sentiment_label`, `confidence_score`, `model_version_id` FK)
- **Product**: Danh sách sản phẩm theo ASIN
- **MLModel**: Thông tin phiên bản model
- **StreamTopic**: Metadata về Kafka topics

2.2.4 BFF API Endpoints

Express.js BFF (port 3001) cung cấp 4 endpoint:

Endpoint	Params	Mô tả
GET /api/products	—	Danh sách ASIN
GET /api/overview	<code>dateRange</code> , <code>productAsin</code> , <code>confidenceMin/Max</code>	Thống kê tổng quan
GET /api/trend	<code>dateRange</code> , <code>productAsin</code>	Xu hướng theo ngày
GET /api/reviews	<code>dateRange</code> , <code>productAsin</code> , <code>page</code> , <code>limit</code> , <code>confidenceMin/Max</code>	Danh sách review

2.3 Mô hình RoBERTa

2.3.1 Giới thiệu RoBERTa

RoBERTa (Robustly Optimized BERT Pretraining Approach) là mô hình ngôn ngữ lớn được phát triển bởi Facebook AI, cải tiến từ BERT với các kỹ thuật:

- Huấn luyện lâu hơn với batch size lớn hơn
- Loại bỏ next sentence prediction objective
- Huấn luyện trên tập dữ liệu lớn hơn (160GB text)
- Dynamic masking thay vì static masking

2.3.2 Model được sử dụng

Đồ án sử dụng checkpoint `cardiffnlp/twitter-roberta-base-sentiment` từ Hugging Face:

- Fine-tuned trên 58 triệu tweet từ Twitter
- 3 nhãn đầu ra: **positive**, **neutral**, **negative**
- Input tối đa 512 token
- Tokenizer tương ứng: `AutoTokenizer` với `normalization=True`

2.3.3 Quy trình inference

1. Đọc message từ Kafka topic `processed_comments`
2. Tokenize văn bản với `AutoTokenizer`
3. Chạy forward pass qua mô hình, lấy logits
4. Áp dụng softmax để ra xác suất 3 nhãn
5. Chọn nhãn có xác suất cao nhất làm `sentiment_label`
6. Giá trị xác suất cao nhất là `confidence_score`
7. Publish kết quả vào topic `final_comments`

Để tối ưu hiệu suất, `model.py` xử lý theo batch nhỏ (default 8 messages/batch) và chạy inference trên CPU hoặc GPU tùy môi trường.

3 Giải pháp và triển khai

3.1 Môi trường và công cụ

3.1.1 Hạ tầng

Toàn bộ hạ tầng được container hóa bằng Docker Compose, bao gồm:

Service	Image	Port
Zookeeper	confluentinc/cp-zookeeper:7.4.0	2181
Kafka	confluentinc/cp-kafka:7.4.0	29092 (host), 9092 (internal)
Flink JobManager	flink:1.17-scala_2.12	8081
Flink TaskManager	flink:1.17-scala_2.12	—
PostgreSQL	postgres:15	5432

3.1.2 Công cụ phát triển

- **Python 3.10+**: Data ingestion, sentiment model (pip packages: kafka-python, transformers, torch, psycopg2, fastapi)
- **Java 11 / Gradle**: Apache Flink job (shadowJar để đóng gói fat JAR)
- **Node.js 18+**: BFF Express.js (express, pg, cors, dotenv)
- **React 18 + Vite**: Frontend dashboard (axios, recharts)

3.2 Data Ingestion

3.2.1 Producer

Script `data_ingestion/producer.py` đọc dữ liệu từ các file trong `raw_data/` và đẩy vào Kafka:

- Hỗ trợ nhiều định dạng file: JSON (Amazon reviews), CSV (sentiment140, train)
- Multiplexing round-robin ngẫu nhiên giữa các file để tạo luồng đa dạng
- Serialize mỗi record thành JSON trước khi publish vào topic `social_media_stream`
- Kết nối Kafka qua `localhost:29092` (host network)

3.2.2 Worker

Script `data_ingestion/worker.py` đọc từ topic `final_comments` và lưu vào PostgreSQL:

- Consumer group riêng biệt để không ảnh hưởng model
- Insert vào bảng `AmazonReview` và `SentimentResult` trong cùng transaction
- Kiểm tra FK constraint với bảng `MLModel` (seed row phải tồn tại trước)
- Xử lý duplicate bằng `ON CONFLICT DO NOTHING` theo `review_id`

3.3 Xử lý luồng với Apache Flink

Flink job được đóng gói thành fat JAR và submit qua Flink Dashboard (<http://localhost:8081>):

1. **Source**: `KafkaSource<String>` đọc từ topic `social_media_stream`
2. **Deserialization**: Parse JSON thành `SocialMediaComment` (timestamp, textComment, topic, label)
3. **Map transformation**: Hàm `cleanText()` áp dụng regex:

- Loại URL: `https?://\S+`
- Loại @mention: `@\w+`
- Loại #hashtag: `#\w+`
- Chuẩn hóa khoảng trắng

4. **Sink:** `KafkaSink<String>` ghi vào topic `processed_comments`

Hai địa chỉ Kafka được dùng: `kafka:9092` cho Flink (Docker network) và `localhost:29092` cho Python scripts (host).

3.4 Phân tích cảm xúc

Script `model/model.py` chạy liên tục như một Kafka consumer-producer:

1. Khởi tạo `AutoModelForSequenceClassification` và `AutoTokenizer` từ checkpoint `cardiffnlp/twitter-roberta-base-sentiment-latest`
2. Lắng nghe topic `processed_comments` (consumer group: `sentiment-group`)
3. Với mỗi message: `tokenize` → `inference` → `softmax` → lấy label và confidence
4. Đóng gói kết quả cùng metadata gốc (`asin`, `star_rating`, `event_time`) thành JSON
5. Publish vào topic `final_comments`

Để phát hiện crash sớm trong quá trình khởi động, script `start-demo.sh` dùng vòng lặp kiểm tra log file cho tín hiệu "`consumer started`" với timeout và crash detection.

3.5 BFF và Dashboard

3.5.1 Backend For Frontend (BFF)

Express.js BFF tại port 3001 đóng vai trò trung gian giữa React và PostgreSQL:

- Parameterized queries ngăn chặn SQL injection
- Filter confidence score server-side (không lọc phía client)
- CORS enabled để frontend dev server có thể gọi API
- Proxy trong `vite.config.js`: `/api` → `http://localhost:3001`

3.5.2 Dashboard React

Giao diện được xây dựng bằng React 18 + Vite, sử dụng Recharts cho biểu đồ:

- **FilterBar:** Chọn khoảng thời gian và lọc theo ASIN sản phẩm
- **OverviewCards:** 4 thẻ thống kê — tổng review, độ tin cậy trung bình, % positive/neutral/negative
- **SentimentPieChart:** Biểu đồ tròn 3 phần với nhãn phần trăm
- **TrendChart:** Biểu đồ cột xu hướng theo ngày (positive/neutral/negative)
- **ReviewTable:** Bảng 50 review/trang với filter "Chỉ xem Tiêu cực" và filter độ tin cậy (70–80%, 80–90%, 90–100%)

Auto-refresh 5 giây để cập nhật dữ liệu theo thời gian thực.

3.5.3 Confidence Filter

Filter độ tin cậy được truyền từ frontend xuống BFF dưới dạng `confidenceMin/confidenceMax`, áp dụng vào cả endpoint `/overview` (cập nhật pie chart) lẫn `/reviews` (cập nhật bảng):

```
AND ($3::numeric IS NULL OR sr.confidence_score >= $3)
AND ($4::numeric IS NULL OR sr.confidence_score <= $4)
```

3.6 Kết quả demo

3.6.1 Khởi động hệ thống

Script `start-demo.sh` tự động hóa toàn bộ quy trình:

1. Khởi động Docker Compose (Kafka, Flink, PostgreSQL)
2. Chờ Kafka sẵn sàng và tạo các topic cần thiết
3. Submit Flink job qua REST API
4. Khởi động `model.py` và chờ consumer sẵn sàng
5. Khởi động `producer.py` và `worker.py`
6. Khởi động BFF và Frontend dev server

3.6.2 Kết quả

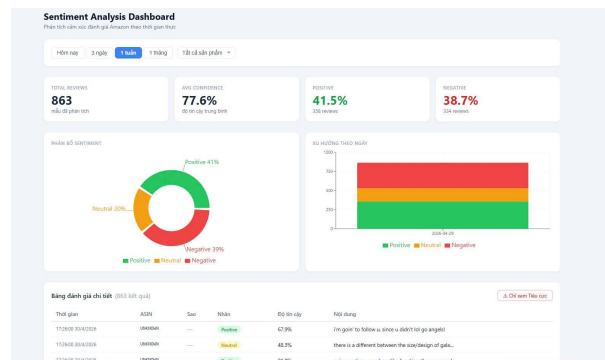


Figure 2: Dashboard hiển thị

Dashboard hiển thị:

- Số lượng review theo thời gian thực tăng dần
- Phân bố cảm xúc: positive chiếm đa số với các review Amazon (sản phẩm điện thoại/phụ kiện)
- Bảng review cho phép click vào từng row để xem nội dung đầy đủ
- Filter confidence giúp tập trung vào các dự đoán có độ tin cậy cao

4 Hướng phát triển

Hệ thống hiện tại đã chứng minh tính khả thi của pipeline phân tích cảm xúc theo thời gian thực. Tuy nhiên, để triển khai ở quy mô sản xuất thực tế, còn nhiều hướng cải tiến đáng được nghiên cứu thêm.

4.1 Mở rộng quy mô (Horizontal Scaling)

Kiến trúc hiện tại chạy trên một máy duy nhất (single node). Để xử lý hàng triệu review mỗi giờ, hệ thống cần được mở rộng theo chiều ngang:

- **Kafka cluster:** Tăng số lượng broker và partition để tăng thông lượng. Mỗi partition có thể được xử lý song song bởi một consumer khác nhau, giúp giảm độ trễ tổng thể tuyến tính theo số máy.
- **Flink cluster:** Thêm nhiều TaskManager để tăng degree of parallelism. Flink hỗ trợ auto-scaling với Kubernetes operator, tự động thêm/bớt worker theo tải.
- **Model inference:** Triển khai nhiều instance `model.py` chạy song song, mỗi instance đọc từ một partition Kafka riêng. Kết hợp với GPU cluster (NVIDIA Triton Inference Server) để tăng tốc inference RoBERTa lên 10–100x so với CPU.
- **PostgreSQL:** Chuyển sang giải pháp phân tán như CockroachDB hoặc dùng read replica để tách tải đọc/ghi.

4.2 Tích hợp nguồn dữ liệu thực

Hiện tại hệ thống đọc từ file tĩnh trong `raw_data/`. Để phân tích dữ liệu thực, cần tích hợp với các API chính thức:

- **Amazon Product Advertising API / SP-API:** Lấy review sản phẩm theo thời gian thực từ Amazon Marketplace. API cung cấp webhook khi có review mới, loại bỏ cần polling.
- **Meta Graph API (Facebook/Instagram):** Thu thập comment, post công khai về sản phẩm từ fanpage doanh nghiệp. Cần xử lý OAuth 2.0 và tuân thủ chính sách dữ liệu của Meta.
- **Twitter/X API v2:** Theo dõi mention, hashtag liên quan đến thương hiệu theo thời gian thực qua Filtered Stream endpoint.
- **Google Play / App Store Reviews:** Sử dụng các thư viện scraping hoặc API không chính thức để thu thập review ứng dụng di động.

Mỗi nguồn dữ liệu cần một connector riêng biệt nhưng đều hướng tới cùng một Kafka topic, giúp pipeline downstream không thay đổi.

4.3 Cải thiện hiển thị — Line Chart theo thời gian thực

Dashboard hiện tại dùng biểu đồ cột (bar chart) theo ngày. Để theo dõi xu hướng tức thời, cần cải tiến:

- **Line chart theo giây/phút:** Hiển thị 3 đường positive/neutral/negative trên trục thời gian rolling window (ví dụ 5 phút gần nhất). Thách thức kỹ thuật là group-by theo giây cần endpoint riêng với query dạng `date_trunc('second', event_time)`.
- **WebSocket thay polling:** Thay cơ chế auto-refresh 5 giây bằng WebSocket (Socket.io) để BFF push dữ liệu ngay khi có review mới, giảm độ trễ hiển thị xuống dưới 1 giây.
- **Sliding window:** Dùng Flink windowing (tumbling/sliding window) để tính aggregate liên tục, BFF chỉ cần đọc từ bảng aggregate thay vì tính toán trên toàn bộ raw data.

4.4 Sửa lỗi và chuẩn hóa thời gian

Hệ thống hiện tại có một số vấn đề liên quan đến timestamp:

- **Timezone inconsistency:** Timestamp từ dataset Amazon được lưu theo UTC nhưng dashboard hiển thị theo múi giờ local của trình duyệt, gây nhầm lẫn. Cần chuẩn hóa toàn bộ pipeline về UTC và chuyển đổi tại frontend.
- **Event time vs Processing time:** Flink hiện dùng processing time (thời điểm message đến Flink). Nên chuyển sang event time (thời điểm review được viết) để phân tích xu hướng chính xác hơn, cần xử lý out-of-order events bằng watermark.
- **Fake timestamp:** Producer hiện dùng thời gian hiện tại (`datetime.now()`) thay vì thời gian gốc của review. Cần đọc trường `unixReviewTime` từ dataset Amazon và truyền qua toàn bộ pipeline.

4.5 Dự báo xu hướng bằng AI

Bổ sung lớp dự báo (forecasting) dựa trên lịch sử sentiment:

- **Time-series forecasting:** Dùng mô hình LSTM hoặc Prophet (Facebook) để dự báo tỉ lệ negative trong 1-24 giờ tiếp theo dựa trên dữ liệu lịch sử. Dashboard có thể hiển thị vùng dự báo (confidence interval) trên line chart.
- **Anomaly detection:** Phát hiện bất thường khi tỉ lệ negative tăng đột biến (ví dụ do lỗi sản phẩm hoặc scandal) và gửi cảnh báo qua email/Slack.
- **Topic modeling:** Kết hợp LDA (Latent Dirichlet Allocation) để tự động nhóm các review negative theo chủ đề (giao hàng chậm, sản phẩm lỗi, dịch vụ kém...) giúp doanh nghiệp xác định vấn đề cụ thể cần giải quyết.
- **Multilingual support:** Mở rộng sang các mô hình đa ngôn ngữ như `xlm-roberta-base` để hỗ trợ phân tích review tiếng Việt và các ngôn ngữ khác ngoài tiếng Anh.

4.6 Tăng cường bảo mật và vận hành

- **Authentication:** Thêm JWT authentication cho BFF API, ngăn truy cập trái phép vào dữ liệu nhạy cảm.
- **Monitoring:** Tích hợp Prometheus + Grafana để theo dõi throughput Kafka, latency Flink, và resource usage của model.
- **CI/CD:** Tự động hóa build, test, và deploy bằng GitHub Actions. Mỗi lần cập nhật model, pipeline tự động rebuild container và deploy không downtime (rolling update).
- **Data retention:** Định nghĩa chính sách xóa dữ liệu cũ (ví dụ giữ tối đa 90 ngày) để tránh PostgreSQL phình to theo thời gian.

Tài nguyên và Mã nguồn

GitHub Repository

Toàn bộ mã nguồn của đồ án được lưu trữ công khai tại:

<https://github.com/fathenot/BigData-L01>

Repository bao gồm:

- `data_ingestion/` — Producer, worker, và API server
- `model/` — RoBERTa sentiment analysis pipeline
- `bff/` — Express.js Backend For Frontend
- `frontend/` — React dashboard (Vite + Recharts)
- `src/` — Apache Flink job (Java)
- `database/` — Schema SQL và migration scripts
- `docker-compose.yml` — Toàn bộ hạ tầng container
- `start-demo.sh` / `stop-demo.sh` — Script khởi động/dừng tự động
- `document/` — Báo cáo LaTeX (file này)

Hướng dẫn chạy

```
# Clone repository
git clone https://github.com/fathenot/BigData-L01.git
cd BigData-L01
```

```
# Khởi động toàn bộ pipeline
bash start-demo.sh
```

```
# Truy cập dashboard
# http://localhost:5173
```

```
# Dừng hệ thống
bash stop-demo.sh
```

Tài liệu tham khảo

1. Y. Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv:1907.11692*, 2019.
2. F. Barbieri et al., “TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification,” *EMNLP Findings*, 2020. (cardiffnlp/twitter-roberta-base-sentiment)
3. Apache Software Foundation, “Apache Flink — Stateful Computations over Data Streams,” <https://flink.apache.org>, 2024.
4. Apache Software Foundation, “Apache Kafka,” <https://kafka.apache.org>, 2024.
5. Hugging Face, “Transformers: State-of-the-art Machine Learning,” <https://huggingface.co/docs/transformers>, 2024.
6. Ni, J. et al., “Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects,” *EMNLP*, 2019. (Amazon review dataset)